

Project Documentation: Serial-In Parity Generator Unit

Digital Design Lab

Project Overview

This design implements a parameterized **Serial Parity Generator** in Verilog HDL[cite: 29]. The unit receives serial data bit-by-bit through `serialin` and shifts it into an internal parallel register (`shift_reg`) on each rising clock edge[cite: 29]. A Verilog function (`calcparity`) calculates the odd/even parity across the registered parallel frame using the XOR reduction operator (`^`)[cite: 29].

Specifications & Functionality

- **Register Width (w):** Parameterized to 8 bits by default[cite: 29].
- **Synchronous Reset:** Resets the internal shift register contents to zero on `posedge clk` when `rst = 1`[cite: 29].
- **Serial Shifting:** Shifts existing bits left and appends `serialin` to the LSB on each `posedge clk`[cite: 29].
- **Combinational Parity Calculation:** Calculates output parity continuously via `assign parity_out = calcparity(shift_reg)`[cite: 29].

Port Summary

Port Name	Direction	Type	Width	Description
clk	Input	wire	1	Clock input signal[cite: 29, 30]
rst	Input	wire	1	Synchronous active-high reset[cite: 29, 30]
serialin	Input	wire	1	Serial data bit input stream[cite: 29, 30]
parity_out	Output	wire	1	Calculated parity bit output[cite: 29, 30]

Table 1: Parity Generator Port Interfaces

Verilog Source Code

Design Module (parity_out.v)

```
module parity_out #(
    parameter w = 8
) (
    input wire clk,
    input wire rst,
    input wire serialin,
    output wire parity_out
```

```

);

    reg [w-1:0] shift_reg;

    // Verilog Function to calculate reduction XOR parity
    function calcparity;
        input [w-1:0] datain;
        begin
            calcparity = ^datain;
        end
    endfunction

    // Synchronous shifting process
    always @(posedge clk) begin
        if (rst) begin
            shift_reg <= {w{1'b0}};
        end else begin
            shift_reg <= {shift_reg[w-2:0], serialin};
        end
    end

    // Continuous assignment calling function
    assign parity_out = calcparity(shift_reg);

endmodule

```

Testbench Module (parity_test.v)

```

`timescale 1ns / 1ps

module parity_test;

    localparam w = 8;

    reg clk;
    reg rst;
    reg serialin;
    wire parity_out;

    parity_out #(
        .w ( w )
    ) dut (
        .clk      ( clk      ),
        .rst      ( rst      ),
        .serialin  ( serialin ),
        .parity_out ( parity_out )
    );

    initial clk = 0;
    always #5 clk = ~clk;

    task serialinn (
        input [7:0] enter,
        input      expected
    );
        integer i;

```

```
begin
  for (i = 7; i >= 0; i = i - 1) begin
    @(negedge clk);
    serialin = enter[i];
  end

  @(posedge clk);
  #1;
  if (parity_out == expected) begin
    $display("Test passsed with frame = %b, output parit is %b",
      enter, expected);
  end else begin
    $display("Test failed= %b output parit is not as expected",
      enter);
  end
end
end
endtask

initial begin
  rst = 1;
  serialin = 0;

  #20;
  rst = 0;

  serialinn(8'b00000111, 1'b1);
  serialinn(8'b11100111, 1'b0);
  serialinn(8'b01110111, 1'b0);

  #20;
  $stop;
end

endmodule
```

Simulation & Verification

Execution Command

To run batch simulation using Cadence Xcelium or ModelSim:

```
xrun parity_out.v parity_test.v
```

Expected Simulation Output

The simulation verifies serial transmission frames and calculates expected parity bits[cite: 30]:

```
Test passsed with frame = 00000111, output parit is 1
Test passsed with frame = 11100111, output parit is 0
Test passsed with frame = 01110111, output parit is 0
```